You're right: I gave two different groupings. Here's the corrected plan where **Milestones = contiguous commit ranges**. One commit can still map to one issue/card, but the milestones now line up cleanly.

---

# Milestones mapped to commits

### Milestone M0 — Baseline

- **Commit 0**: `chore: baseline snapshot before studio runtime work`

---

### Milestone M1 — Thread Memory (Server-side)

Goal: you can chat with a persistent `thread_id` and export the thread.

- **Commit 1**: `chore: normalize package structure for CBRA runtime`
- **Commit 3**: `feat: add thread, message, loopcycle data models`
- **Commit 4**: `feat: in-memory thread store with create/get/append`
- **Commit 5**: `feat: add /chat and /thread endpoints for server-side memory`
- **Commit 6**: `feat: normalize response envelope with request_id and thread_id`

*(Commit 2 is corpus; commit 7 is kernel; those belong in M2.)*

---

### Milestone M2 — Canon + Kernel Lock

Goal: runtime loads canonical corpus and kernel is non-overridable.

- **Commit 2**: `feat: load CBRA canonical corpus files from manifest`
- **Commit 7**: `feat: enforce locked CBRA kernel loading from canonical file`

---

### Milestone M3 — LoopCycle Debug Contract + Drift Contract

Goal: debug aligns to loop stages; drift exists as a structured event with reintegration target (even if stage runner isn't fully enforced yet).

- **Commit 8**: `feat: emit LoopCycle-shaped debug aligned to Loops Core stages`
- **Commit 9**: `feat: drift event logging scaffold and reintegration hooks`

---

### Milestone M4 — Studio UI

Goal: browser UI uses `thread_id`, renders markdown, shows per-turn debug, exports.

- **Commit 10**: `feat: studio chat UI with thread persistence and debug panel`

---

## GitHub Project Board structure (exact)

Columns:

- Backlog
- In Progress
- Review/Test
- Done

Create **one issue per commit**, and label each issue with the milestone label:

- `M0` → Commit 0
- `M1` → Commits 1,3,4,5,6
- `M2` → Commits 2,7
- `M3` → Commits 8,9
- `M4` → Commit 10

Suggested issue titles = commit messages.

---

## Why commit numbers skip "Commit 2" in M1

Because “canonical corpus wiring” is independent of thread memory. You can ship M1 without corpus changes. That’s intentional sequencing: memory first, canon lock second.

---

If you want, I can rewrite the commit list itself so it’s strictly sequential without the earlier “option A/B” fork (i.e., a single canonical path with exact filenames).